Reviewer Pack — Plethysm Coefficient Exponential Gap in Symmetric Squares of...

Entry c93f42c3f6f7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 03:07:29 UTC

Plethysm Coefficient Exponential Gap in Symmetric Squares of Permutation Representations for Random CNFs

Entry ID: `c93f42c3f6f7` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 03:07:29 UTC

1. Conjecture

Field A (mathematical branch): Plethysm Theory **Field B** (complexity object): Boolean Circuit Size

Statement:

> For a random 3-CNF formula Φ with n variables and m clauses, let π(Φ) denote the plethysm coefficient of its characteristic polynomial under the symmetric square representation of S_n. Then π(Φ) ≥ 2^{n/2} · poly(m) with probability ≥ 2/3, while the determinant's symmetric square plethysm coefficient is bounded by poly(n,m).

Rationale (proposer's reasoning):

> Plethysm coefficients capture the multiplicity of irreducible representations in symmetric power decompositions. The permanent's higher multiplicities in symmetric squares may reflect its inherent complexity, while the determinant's bounded coefficients suggest structural simplicity. This gap could imply circuit size separations via representation-theoretic obstructions.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `63590bd25f32d6b1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from itertools import combinations, permutations

def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

def generate_primes(n=30):
    primes = []
    num = 2
    while len(primes) < n:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes

def random_3cnf(n, m):
    variables = list(range(1, n + 1))
    clauses = set()
    for _ in range(m):
        clause = tuple(random.sample(variables, 3))
        if len(clause) == 3:
            clauses.add(tuple(sorted(clause)))
    return clauses

def generate_partitions(n):
    def partition_helper(n, max_val, current_partition):
        if n == 0:
            yield current_partition
        else:
            for i in range(1, min(max_val, n) + 1):
                yield from partition_helper(n - i, i, current_partition + [i])

    return list(partition_helper(n, n, []))
```

```
def hook_length_formula(shape, n):
    total = 1
    for row in shape:
        for col in range(row):
            total *= (n - row + col + 1)
        total //= (row * (col + 1))
    return total

def plethysm_coefficient(char_poly, n):
    partitions = generate_partitions(n)
    return sum(hook_length_formula(shape, n) * char_poly[shape] for shape in partitions)

def characteristic_polynomial(clauses, n):
    char_poly = {(): 1}
    for clause in clauses:
        new_char_poly = {}
        for partition, coeff in char_poly.items():
            for i in range(n + 1):
                if len(partition) + i <= n:
                    new_partition = tuple(sorted(partition + (i,) * len(clause)))
                    new_char_poly[new_partition] = new_char_poly.get(new_partition, 0) + coeff
        char_poly = new_char_poly
    return char_poly

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10
    m = 40
    clauses = random_3cnf(n, m)

    char_poly = characteristic_polynomial(clauses, n)
    plethysm = plethysm_coefficient(char_poly, n)

    conjecture_holds = plethysm >= 2**(n/2) * (m + 1)**(n/2)
    counterexample = "" if conjecture_holds else "plethysm_too_small"

    return {
        "metric_name": "plethysm_coefficient",
        "metric_value": plethysm,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        primes = generate_primes()
        seeds = primes[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```
    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = (sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))**0.
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        counterexample_desc = "plethysm_too_small"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample_desc}\" first_failing_seed={fir
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_258cdfd5.py", line 105, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_258cdfd5.py", line 84, in run_trial
    plethysm = plethysm_coefficient(char_poly, n)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_258cdfd5.py", line 63, in plethysm_coef
    return sum(hook_length_formula(shape, n) * char_poly[shape] for shape in partitions)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_258cdfd5.py", line 63, in <genexpr>
    return sum(hook_length_formula(shape, n) * char_poly[shape] for shape in partitions)
                                               ~~~~~~~~~^^^^^^^
TypeError: unhashable type: 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to unhashable type error, preventing data collection. Cannot assess conjecture validity without successful runs. | next: Fix the char_poly data structure to use a dictionary instead of list for hashable lookups

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111389
2	propose	ollama_remote	qwen3:8b	0	0	36959
3	preregistration	ollama_remote	qwen3:8b	0	0	24324
4	novelty	ollama_remote	qwen3:8b	0	0	20704
5	novelty	ollama_remote	qwen3:8b	0	0	17548
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17213
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11042
8	judge	ollama_remote	qwen3:8b	0	0	14962

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 254140 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/c93f42c3f6f7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c93f42c3f6f7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c93f42c3f6f7.tar.gz` (if generated)